

Pandas Coding Questions

Core Data Structures

Question 1

Create a Pandas Series containing the values `10`, `20`, `30`, `40` using the default index.

Hint: Use `pd.Series()` with a list of values. Do not specify an index.

Question 2

Create a Pandas Series containing the values `100`, `200`, `300` with custom index labels `a`, `b`, and `c`.

Hint: Use `pd.Series()` and provide custom labels using the `index` parameter.

Question 3

Create a DataFrame with columns `Name` and `Age` using a dictionary.

Hint: Create a dictionary with keys as column names and pass it to `pd.DataFrame()`.

Question 4

Given a DataFrame `df`, display its row index, column names, and shape.

Hint:

- Row index → `df.index`
- Column names → `df.columns`
- Shape → `df.shape`

Creating DataFrames

Question 5

Create a DataFrame from the following list of lists and assign appropriate column names:

```
[
    ["Amit", 21],
    ["Riya", 22],
    ["John", 23]
]
```

Hint: Use `pd.DataFrame()` with the list of lists and specify column names using the `columns` parameter.

Question 6

Create a DataFrame from a NumPy array containing student marks and assign column names.

Hint: Create a NumPy array first, then convert it into a DataFrame using `pd.DataFrame()` and assign column names.

Question 7

Read a CSV file named `students.csv` into a Pandas DataFrame.

Hint: Use the Pandas function for reading CSV files.

Question 8

Read a JSON file named `students.json` into a Pandas DataFrame.

Hint: Use the Pandas function for reading JSON files.

Question 9

Display the first 5 rows, last 5 rows, DataFrame information, and column names of a DataFrame.

Hint:

- First 5 rows → `head()`
- Last 5 rows → `tail()`
- Information → `info()`
- Column names → `columns`

Question 10

Find the number of rows and columns present in a DataFrame.

Hint: Use the DataFrame attribute that returns rows and columns together.

Data Selection & Filtering

Question 11

Select the `Name` column from a DataFrame and check its type.

Hint: Select a single column using square brackets and the column name.

Question 12

Select the `Name` and `Age` columns from a DataFrame.

Hint: Use double square brackets and provide a list of column names.

Question 13

Retrieve a row using its label with `loc`.

Hint: Use `loc` with the row label.

Question 14

Retrieve the third row of a DataFrame using `iloc`.

Hint: Use `iloc` with the row position. Remember indexing starts from `0`.

Question 15

Access a specific cell value using row label and column name.

Hint: Use `loc[row_label, column_name]`.

Question 16

Access a specific cell value using row and column positions.

Hint: Use `iloc[row_position, column_position]`.

Question 17

Select a range of rows and specific columns using `loc`.

Hint: Use `loc` with a row label range and a list of column names.

Question 18

Select a range of rows and columns using `iloc`.

Hint: Use `iloc` with row and column position ranges.

Question 19

Access a single value from a DataFrame using `at`.

Hint: Use `at[row_label, column_name]` for accessing a single value.

Question 20

Access a single value from a DataFrame using `iat`.

Hint: Use `iat[row_position, column_position]` for accessing a single value.

Filtering Data

Question 21

Filter all movies with an IMDb rating greater than 8.

Hint: Create a Boolean condition using the `IMDb` column and filter the DataFrame.

Question 22

Filter all movies where the IMDb rating is greater than 8 OR the year is greater than 2020.

Hint: Combine two conditions using the OR operator (`|`). Wrap each condition in parentheses.

Question 23

Filter all movies with an IMDb rating greater than 8 using the `query()` method.

Hint: Use `query()` and write the condition as a string.

Question 24

Create a copy of selected columns from a DataFrame and modify one value without affecting the original DataFrame.

Hint:

1. Select the required columns.
2. Create a copy using `.copy()` .
3. Modify a value using `loc` .
4. Display the copied DataFrame.

Question 25

Demonstrate the difference between a view and a copy in Pandas.

Hint:

1. Create one subset without `.copy()` .
2. Create another subset using `.copy()` .
3. Modify values in both subsets.
4. Compare them with the original DataFrame and observe the behavior.